

Guia de Sobrevivência: Listas 03 e 04

Resolução Passo a Passo e Práticas de Programação

Aviso aos Alunos: Este material foi preparado para guiar vocês no mapeamento de conceitos da disciplina de Análise e Projeto Orientado a Objetos diretamente para o código Java prático. Aqui vocês encontrarão o passo a passo resolvido e comentado das nossas missões educacionais, para garantir que dominem a arquitetura de pacotes e os princípios de encapsulamento sem dificuldades.

Lista 03 (Missão 02): O Arquiteto de Módulos (Classes e Pacotes)

Nesta etapa, o objetivo é abandonar a ideia de colocar todo o código em um "arquivo único" e construir um sistema estruturado onde duas classes conversam entre si [cite: 1]. No mercado corporativo, um sistema separa suas responsabilidades em "bairros" (pacotes lógicos), para que as regras de negócio fiquem devidamente isoladas [cite: 1].

Passo 1: Preparando a Fundação (Local)

Vamos criar o projeto na máquina local antes de enviá-lo para a nuvem [cite: 1]. Siga a sequência de comandos no seu Git Bash:

```
cd ~/Desktop
mkdir meu-primeiro-sistema
cd meu-primeiro-sistema
git init
```

Entendendo o Porquê

A sequência cria a pasta base para o projeto (`mkdir`) e, em seguida, liga o "Motor do Git" (`git init`) para que a ferramenta comece a rastrear todas as alterações feitas nessa pasta [cite: 1].

Passo 2: A Arquitetura de Pastas (O Domínio)

A linguagem Java exige que a estrutura de pastas siga um formato de domínio invertido [cite: 1]. Com o terminal ainda aberto, digite:

```
mkdir -p src/br/com/meusistema/model
mkdir -p src/br/com/meusistema/main
```

O que isso resolve: Você acaba de criar a pasta raiz `src` (Source), que guarda `br/com/meusistema`, dividida em dois pacotes essenciais: `model` (que guardará as plantas/moldes das classes estruturais) e `main` (responsável pela execução do programa) [cite: 1].

Passo 3: Criando os "Tijolos" (A Classe Carro)

No terminal, abra a sua IDE utilizando o comando `code` . [cite: 1]. Pelo menu lateral, navegue até o pacote `src/br/com/meusistema/model` e crie um novo arquivo chamado `Carro.java` [cite: 1]. Esta é a planta da nossa classe:

```
package br.com.meusistema.model;

public class Carro {
    public String modelo;
    public String cor;

    public void buzinar() {
        System.out.println("O " + modelo + " " + cor + " fez: BIIII BIIII!");
    }
}
```

Sempre pressione `Ctrl+S` para salvar o arquivo após digitar [cite: 1].

Passo 4: Criando o "Motorista" (A Classe Principal)

O molde de Carro não faz nada sozinho, precisamos executá-lo. Navegue até o pacote `src/br/com/meusistema/main` e crie o arquivo `Sistema.java` [cite: 1]:

```
package br.com.meusistema.main;

// 🚨 ATENÇÃO: O Sistema precisa avisar que usará a classe Carro localizada no outro
// pacote! [cite: 1]
import br.com.meusistema.model.Carro;

public class Sistema {
    public static void main(String[] args) {
        System.out.println("Iniciando o sistema...");

        // Criando e instanciando o objeto Carro 1 [cite: 1]
        Carro meuCarro = new Carro();
        meuCarro.modelo = "Fusca";
        meuCarro.cor = "Azul";
        meuCarro.buzinar(); // Chamando o comportamento do objeto [cite: 1]
    }
}
```

✅ Resposta do Desafio: Como criar o Carro 2?

Para resolver o desafio e criar um segundo carro com características diferentes, adicione as linhas abaixo no final do método `main`:

```
Carro carro2 = new Carro();
carro2.modelo = "Gol";
carro2.cor = "Prata";
carro2.buzinar();
```

Passos 5 e 6: Teste de Compilação e Envio para o GitHub

Para compilar todas as pastas de uma só vez pelo terminal, utilize o comando: `javac src/br/com/meusistema/model/*.java src/br/com/meusistema/main/*.java` [cite: 1]. Depois, execute com: `java -cp src br.com.meusistema.main.Sistema` [cite: 1].

Ao ver a mensagem do Fusca azul, o teste está completo [cite: 1]! O passo final é criar um repositório no GitHub (lembre-se de deixá-lo totalmente vazio, sem README) e subir o projeto usando os comandos de versionamento padrão: `git add ., git commit -m "feat: cria`

estrutura de pacotes e classes Carro e Sistema", adição do link remoto e o envio com `git push -u origin main` [cite: 1].

🎯 Lista 04 (Missão 03): O Escudo Protetor (Encapsulamento e .gitignore)

Nesta segunda etapa, a missão principal é proteger os dados do sistema contra entradas ilógicas e configurar a IDE para não enviar arquivos indesejados ao repositório remoto [cite: 2].

Passos 1 e 2: Diagnosticando a Falha e Trancando as Portas

Se você adicionar um atributo `public int velocidade;` no seu Carro, qualquer parte externa do código poderá inserir valores incorretos, como testar um `meuCarro.velocidade = -500;` direto da Main, o que logicamente quebraria a regra do mundo real [cite: 2].

Aplicando o Encapsulamento: O primeiro passo para resolver isso é trancar o acesso direto aos dados. Vá ao arquivo `Carro.java` e altere todos os atributos substituindo `public` por `private` [cite: 2]:

```
private String modelo;  
private String cor;  
private int velocidade;
```

Assim que salvar, a sua classe `Sistema.java` apresentará erros vermelhos [cite: 2]. Não se assuste, isso ocorre porque o sistema perdeu a permissão de tocar nos atributos da classe Carro de forma direta [cite: 2].

Passos 3 e 4: A Mágica da IDE (Getters e Setters)

Com as portas trancadas, precisaremos de métodos públicos (as catracas de acesso) para interagir com os dados [cite: 2]. Não precisamos escrever tudo à mão. Clique com o botão direito no código, vá em "Source Action..." e selecione "Generate Getters and Setters", marcando todas as caixas [cite: 2].

Agora, busque o método gerado `setVelocidade` e adicione a regra de segurança para impedir a inserção de velocidades negativas [cite: 2]:

```
public void setVelocidade(int velocidade) {  
    if (velocidade < 0) {  
        System.out.println("Erro: Velocidade não pode ser negativa!");  
    } else {  
        this.velocidade = velocidade;  
    }  
}
```

Feito isso, vá até o `Sistema.java` e corrija os erros inserindo os dados através das portas corretas: `meuCarro.setModel("Fusca");` e `meuCarro.setCor("Azul");` [cite: 2]. Ao testar `meuCarro.setVelocidade(-500);`, o programa deverá barrar o valor e imprimir a mensagem de erro que você configurou [cite: 2].

Passos 5 a 7: Arquivos Invisíveis e o `.gitignore`

Digitando `git status` no terminal, você notará a presença de arquivos compilados (`.class`) e diretórios próprios da IDE, como o `.vscode/` [cite: 2]. Lembre-se da regra de ouro: **Nós NUNCA mandamos esses arquivos para o GitHub, apenas o código fonte legível (.java)** [cite: 2].

A Resolução: Crie um novo arquivo na raiz do projeto chamado `.gitignore` (com o ponto no início) e digite as instruções de bloqueio exatamente assim [cite: 2]:

```
*.class  
.vscode/  
bin/
```

Teste com `git status` novamente para conferir a mágica acontecendo e os arquivos compilados sumirem da lista do Git [cite: 2]. Agora é só consolidar as alterações executando `git add .`, gerando o commit com a mensagem "feat: aplica encapsulamento e gitignore" e enviando tudo de forma profissional e segura para a nuvem com `git push` [cite: 2].